

C++ & QT Day1-hw3 AI 보고서

로봇 21기 예비단원 최윤서

1. 어느 부분에 AI를 사용했는가

- Attatck(Monster &target) 부분이 잘 이해되지 않아 AI를 사용했습니다.
- AI를 사용하기 전 찾아봤던 내용
 - 1) "class & 연산자"라고 검색하였을 때 연산자 오버로딩에 관한 내용이나 논리연산자에 대한 내용이 많이 나와 정확한 정보를 찾을 수 없었습니다. 이 때 구글링한 정보로 &가 참조 연산자라는 것을 알 수 있었고 그 다음에 C++ 참조 연산자 활용라고 검색해보았습니다.
 - 2) "C++ 참조 연산자 활용"라고 검색했을 때 비교적 정확한 내용이 나왔지만 "함수 이름(int &num)" 이런 형식의 참조 연산자가 많아서 앞에 붙어있는 Monster가 어떤 의미인지를 파악하기 부족했습니다.

2. 깨달은 점

- 결국 call by reference와 같은 원리로 작동한다는 것을 알 수 있었다.
- 따라서 함수 안에서 target을 수정하면 원본 Monster 객체가 실제로 바뀐다
- player.Attack(monster)라고 쓸 수 있는 것도 이와 같은 이유임을 알게되었다.

3. 대화 내용

```
class Player
{
public:
    int HP, MP, A, D;
    Player() {}
    Player(int a, int p):
        HP(a), MP(p) {}
    void Attack(Monster *target):
        Show_Status();
    void Show_Status();
    void Show_HP_Status();
    void Show_MP_Status();
};
```

Attack 함수에서 Monster &target은 어떤걸 의미하는거야?

4시간 전

이미지에 있는 코드를 보면:

```
cpp
void Attack(Monster &target);
```

여기서 **Monster &target** 은 **참조(reference)** 전달을 의미합니다.

&의 의미

C++에서 함수 매개변수 앞에 **&**가 붙으면, 그 매개변수는 ****참조자(reference)****입니다. 즉 값을 복사해서 넘기는 게 아니라, 원본 객체를 그대로 넘긴다는 뜻입니다.

```
cpp
void Attack(Monster &target); // target은 원본 Monster 객체 그 자체
```

값 전달 vs 참조 전달 비교

값으로 전달 (참조 없이):

```
cpp
void Attack(Monster target); // target은 원본의 '복사본'
```

- 함수 안에서 **target**을 수정해도 원본 Monster는 전혀 영향받지 않음
- 객체가 클수록 복사하는 데 비용(메모리, 시간)이 듬

참조로 전달 (& 사용):

```
cpp
void Attack(Monster &target); // target은 원본 Monster의 '별명(alias)'
```

- 함수 안에서 **target**을 수정하면 원본 **Monster 객체가 실제로 바뀜**
- 복사가 일어나지 않아서 효율적

왜 여기서 참조를 쓰는가?

Attack 함수는 몬스터를 공격하는 함수입니다. 공격을 하면 몬스터의 HP가 줄어들어야 하는데, 만약 값으로 전달받으면 함수 안에서 HP를 깎아도 그건 복사본의 HP만 깎이는 것이어서 **원본 몬스터의 HP는 그대로**입니다.

```
cpp
void Player::Attack(Monster &target) {
    target.HP -= 10; // 원본 Monster 객체의 HP가 실제로 감소함
}
```

즉, 실제 게임 로직에서 "이 플레이어가 공격한 그 몬스터"의 상태를 진짜로 바꾸기 위해 참조로 넘기는 것입니다.

4시간 전

main함수에서는 어떻게 활용해야돼?

↓

main 함수에서 활용하는 예시를 보여드릴게요. 다만 `Monster` 클래스의 구체적인 멤버는 이미지에 없으니, 비슷한 구조로 가정해서 작성하겠습니다.

예상되는 Monster 클래스 (가정)

```
cpp
class Monster
{
public:
    int HP, MP, x, y;
    Monster();
    Monster(int x, int y);
    void Show_Status();
};
```

main 함수 예시

```
cpp
#include <iostream>
using namespace std;

int main()
{
    Player player(0, 0);    // 플레이어 생성
    Monster monster(5, 5);  // 몬스터 생성

    monster.Show_Status();  // 공격 전 몬스터 상태 확인

    player.Attack(monster);  // 몬스터를 참조로 넘겨서 공격!

    monster.Show_Status();  // 공격 후 몬스터 상태 확인 (HP가 줄어있어야 함)

    return 0;
}
```

핵심 포인트

```
cpp
player.Attack(monster);
```

여기서 `monster` 라는 객체 자체를 넘기고 있습니다. `&` 를 붙이든 안 붙이든 호출하는 쪽 코드(`main` 함수)에서는 문법이 똑같습니다.

- 함수 선언부(`Attack(Monster &target)`)에 `&` 가 있기 때문에, `main`에서는 그냥 평범하게 `monster`를 넘기면 됩니다.
- 포인터처럼 `&monster` (주소 연산자)를 쓸 필요가 없습니다. 이 부분에서 많이 헷갈리는데, 함수 매개변수의 `&` (참조)와 변수 앞의 `&` (주소 연산자)는 다른 문법입니다.

만약 포인터였다면?

비교를 위해, 만약 함수가 포인터를 받는 형태였다면:

```
cpp
void Attack(Monster *target);  // 포인터로 받는 경우
```

이때는 `main`에서 이렇게 호출해야 합니다:

```
cpp
player.Attack(&monster);  // 주소를 넘겨야 함
```

또는 함수 내부에서 `target->HP` 처럼 화살표 연산자를 써야 하죠.

정리하면: 지금 코드처럼 참조(`&target`)로 선언된 경우, `main`에서는 그냥 일반 객체 변수 이름만 넘기면 되고, `Attack` 함수 내부에서 그 몬스터의 HP를 실제로 변경할 수 있습니다.